

# The Agentic AI Platform, drawn rather than described

Every diagram uses the same six-colour layer key. Learn it once on the next page and the rest of the deck reads itself.



## THE KEY

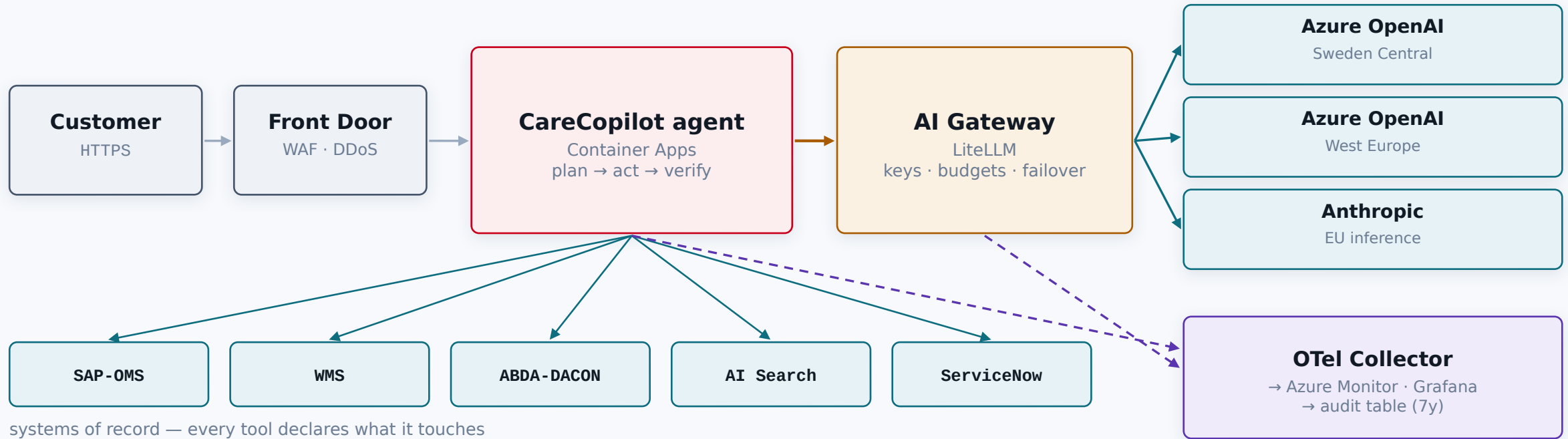
# Six layers, six colours

*Every box in this deck carries one of these colours. It tells you which layer owns the thing, and therefore who is accountable for it.*

|                                                                                                                                           |                                        |                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|  <b>Edge</b><br>Front Door, WAF                          | <b>Everything hostile arrives here</b> | Front Door with a WAF terminates TLS, filters OWASP and geo, and absorbs DDoS before anything reaches the VNet. |
|  <b>Agent</b><br>Container Apps, the loop                | <b>Where the reasoning happens</b>     | Container Apps running the plan/act loop. Holds no provider key and cannot reach the internet.                  |
|  <b>Gateway</b><br>LiteLLM                               | <b>The only path to a model</b>        | LiteLLM. Owns virtual keys, entitlement, rate limits, budgets, failover, caching and cost attribution.          |
|  <b>Data &amp; AI</b><br>OpenAI, Search, Postgres, Redis | <b>Models and knowledge</b>            | Azure OpenAI in two EU regions, AI Search for retrieval, Postgres for spend and transcripts, Redis for cache.   |
|  <b>Observability</b><br>OTel, Monitor, Grafana        | <b>How the platform sees itself</b>    | OTel to Azure Monitor and Grafana. Four signal families, one trace per turn, an append-only audit table.        |
|  <b>Governance</b><br>Entra, Key Vault, policy, HITL   | <b>What keeps it defensible</b>        | Entra ID, Key Vault, policy as code, and the human-approval gate on anything irreversible.                      |

# The whole system

*Solid lines carry a request. Dotted lines carry telemetry. Nothing in the AI plane is reachable from the internet.*



**Key Vault · Entra ID · managed identity**  
no long-lived secret exists anywhere in the chain

**Human approval on side-effecting tools**  
the pharmacist ticket does not exist until someone approves it

● Edge

● Agent

● Gateway

● Data & AI

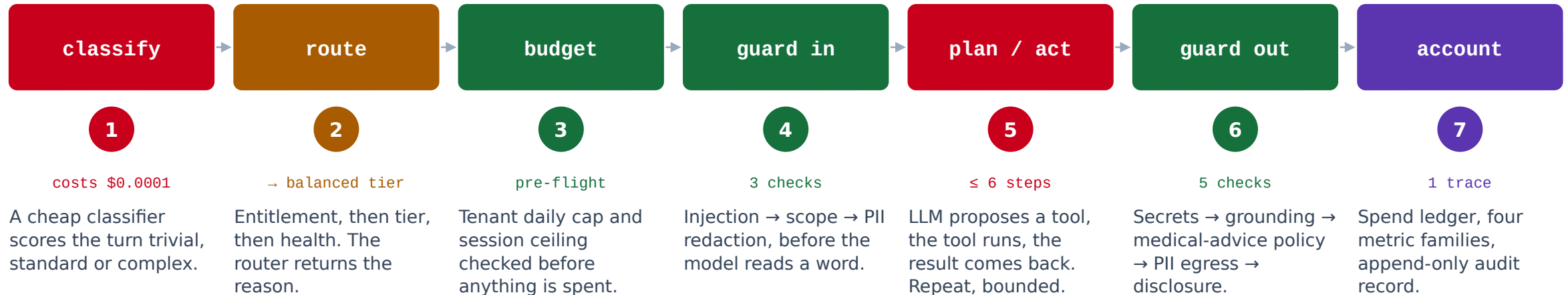
● Observability

● Governance

## THE REQUEST PATH

# One question, every hop

*"Can I take ibuprofen with warfarin?" — and what happens to it, in order.*



### Budget before guardrails

Guardrails cost CPU. A tenant already over budget should be refused before the platform spends anything on them — including its own compute.

### Guardrails before the model

Injection has to be caught before the model reads it, and PII removed before the prompt leaves the VNet. An output-only guardrail already lost.

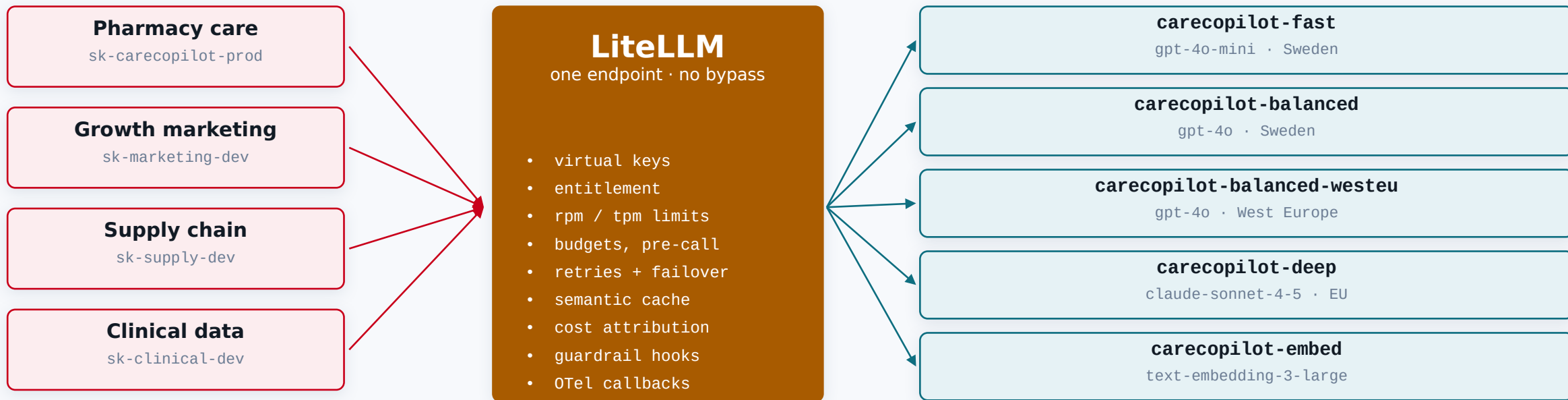
### Approval before the side effect

The agent may read and draft freely. It stops before changing a system of record. That is EU AI Act Art. 14 as a code path.

## THE CHOKE POINT

# Why every call goes through one door

*Without it: N codebases holding provider keys, N retry policies, no cost story, no way to revoke a team's access today.*



Bypassing it is not a policy — it is a network property. The agent subnet denies internet egress and the agent holds no provider key, so it has exactly one route to a model. Tenants ask for 'carecopilot-balanced', never 'gpt-4o-2024-11-20', which is what makes swapping the model underneath a config change rather than a migration.

## ROUTING

# Three gates, in this order, always explained

*Governance first, cost second, reliability third. Reversing any two of these is a different platform.*

### 1 Entitlement

*Is this key allowed this model?*

A governance question, so it goes first. No performance consideration may override it — that is what makes the entitlement real rather than advisory.

### 2 Complexity tier

*Cheapest model that can do the job*

A classifier scores the turn trivial / standard / complex. The mini tier is ~16x cheaper per token; the classifier that decides costs \$0.0001 and pays for itself the first time it keeps a greeting off the deep model.

### 3 Health

*Is the chosen deployment cooling down?*

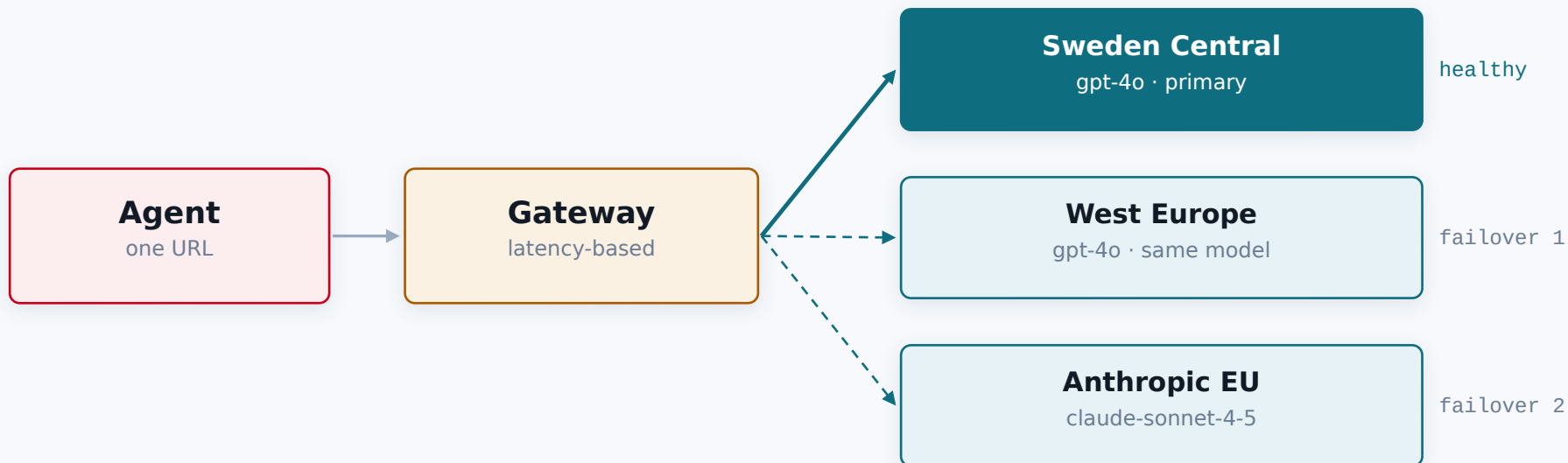
Same model in the second EU region first. A different provider is the last resort, because a provider switch changes behaviour and behaviour changes need an eval run.

#### A decision the trace can explain

```
complexity trivial — greeting with no information need
selected   carecopilot-fast → azure/gpt-4o-mini @ swedencentral
why        classifier scored the turn 'trivial' → downshift to fast tier
```

# What happens when a region goes down

*Two EU regions and two providers. The second region is failover and a second quota pool — quota is allocated per region.*



## Region before provider

Same model in a second region keeps behaviour identical. A provider switch does not — so it is last, and it needs an eval run behind it.

## 3 failures, 60s cooldown

A deployment that fails three times is taken out of rotation for a minute, then probed again. 429s do not count: backpressure is not ill health.

## Context and content fallbacks

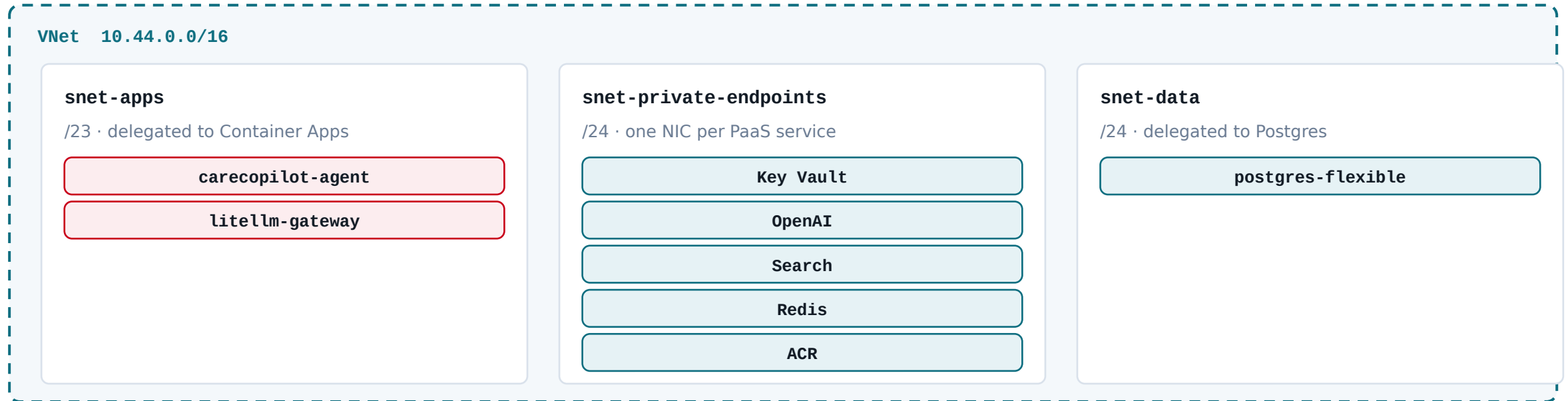
A prompt too long for 128k routes to the 200k model. A provider content-filter block gets a second opinion rather than an error.

A single region is a single point of failure for every AI feature in the company. That is the whole argument for the second one — and it is also where the extra quota comes from.

## TRUST BOUNDARY

# No public ingress, no unmonitored egress

*The rule that shapes every other Azure decision. For a workload handling Art. 9 health data this is the baseline a DPO signs, not hardening.*



### ● Deny inbound from the internet

Ingress only from the Front Door subnet, on 443.

### ● Private endpoint per dependency

Key Vault, OpenAI, Search, Redis, Postgres, ACR, Storage, Monitor.

### ● Deny outbound to the internet

Egress allowed to the AzureCloud service tag only — a compromised agent has nowhere to send data.

### ● A private DNS zone for each one

Without it the endpoint resolves to its public IP and traffic silently leaves the VNet — the most commonly missed control in Azure private networking, and it fails quietly.

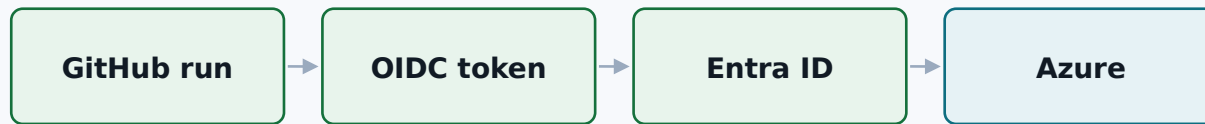


## IDENTITY

# There is no long-lived secret anywhere

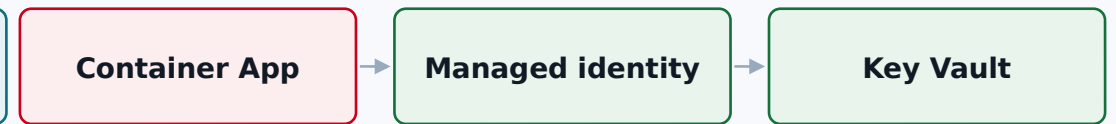
*Two flows, and neither of them ends in a password sitting in a settings page.*

### GitHub Actions → Azure



Entra checks the token's subject against an exact federated credential — a different branch, environment or fork does not match and is rejected before any Azure call. The token lives minutes. Nothing exists to leak or rotate.

### Running app → its secrets



The app holds nothing. It presents a managed identity and Key Vault decides. The gateway and the agent have separate identities, so “the agent was compromised” and “the provider keys were compromised” stay different sentences.

#### ● Deploy identity

Contributor on one resource group. Explicitly barred from granting Owner or User Access Administrator — no privilege-escalation path.

#### ● Agent identity

Key Vault Secrets User · AcrPull · Search Index Data Reader. Cannot read a provider key. Cannot change infrastructure.

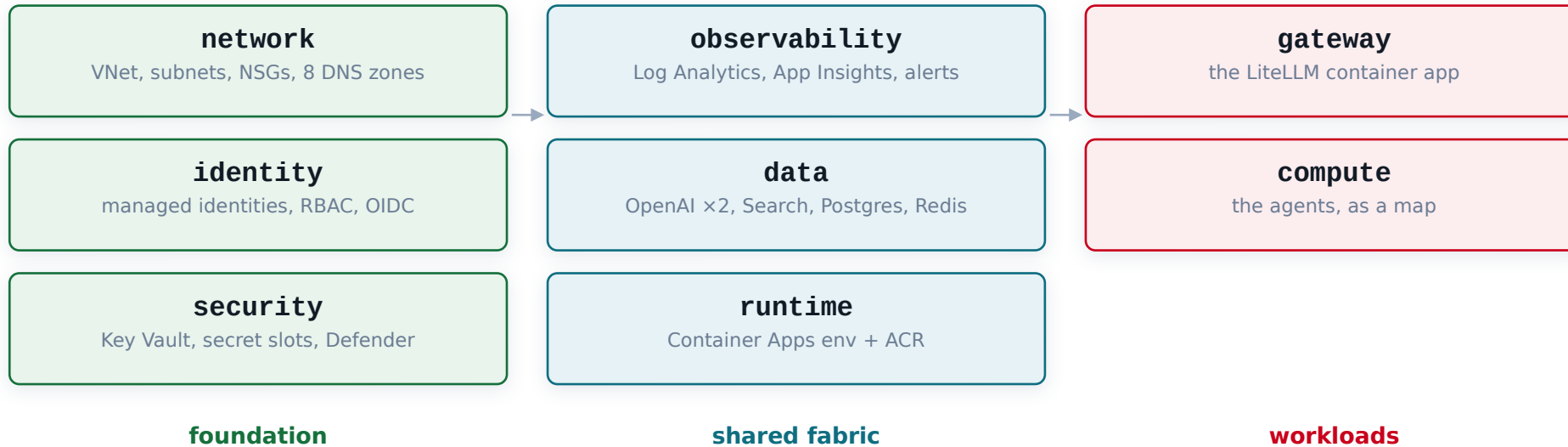
#### ● Gateway identity

The only identity in the platform permitted to call the model endpoints. That is what makes the gateway's governance non-optional.

The tenant holds a virtual key, not a provider key: revocable, budgeted, scoped, and rotatable without redeploying anything.

# The module graph is a design signal

*Eight modules, two environments, one dependency rule — and the bug that proved the rule.*



## The cycle that taught the boundary

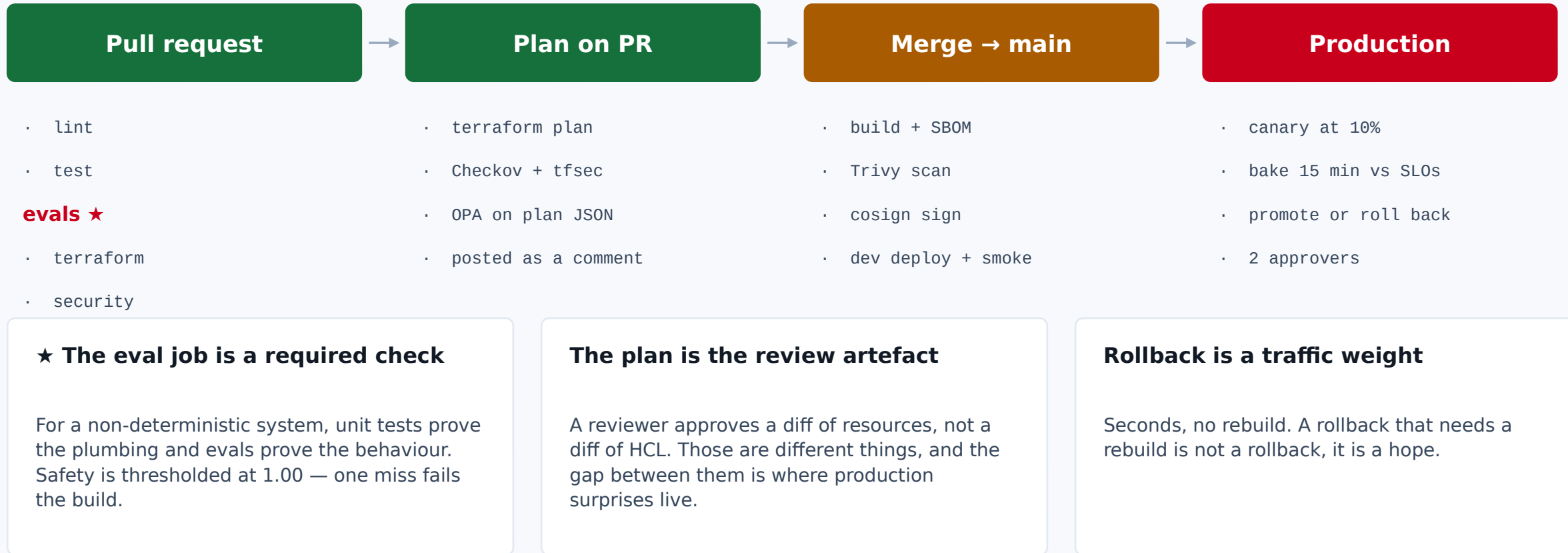
The gateway needs a registry to pull from; the agents need the gateway's URL. Put the registry in compute and the environment in gateway and the two modules depend on each other — Terraform refuses to build a graph at all. The fix was not a workaround: the compute fabric is shared infrastructure both sit on, so it became its own module. Module boundaries follow ownership, not usage.

## dev and prod are the same code

What differs is size and cost: smaller SKUs, no zone redundancy, scale-to-zero. What never differs is posture — private networking, managed identity, no public data access, guardrails on, audit on. If a control had to be switched on for production, dev would be the weak link an attacker uses.

# The only path to production

*Nobody holds standing write access to the Azure subscriptions. The pipeline gets minutes of it, via OIDC.*



## SAFETY

# Two pipelines the tenant never writes

*Shared, so a team does not have to reimplement PII redaction to ship. They get it by adopting the paved road.*

### Input pipeline — before the model reads a word

3 checks

|                               |               |                                    |
|-------------------------------|---------------|------------------------------------|
| <code>prompt_injection</code> | <b>block</b>  | instruction override, exfiltration |
| <code>topic_policy</code>     | <b>block</b>  | outside the pharmacy support scope |
| <code>pii_redaction</code>    | <b>redact</b> | IBAN, email, phone, card, DOB      |

### Output pipeline — before the customer sees it

5 checks

|                             |                 |                                          |
|-----------------------------|-----------------|------------------------------------------|
| <code>secret_leak</code>    | <b>block</b>    | credential-shaped strings in the answer  |
| <code>grounding</code>      | <b>annotate</b> | every claim traced to a tool observation |
| <code>medical_advice</code> | <b>escalate</b> | individualised clinical instruction      |
| <code>pii_egress</code>     | <b>redact</b>   | identifiers on the way back out          |
| <code>disclosure</code>     | <b>annotate</b> | EU AI Act Art. 50, on every reply        |

### The trust boundary that matters most in an agentic system

Tool output is data, never instruction. A policy document, an order note or a product description can carry text engineered to look like a system prompt — that is indirect prompt injection, and it is the hard one. There is no complete defence, so the strategy is layered: detect it, treat tool content as data, and constrain the agent so a successful injection has nowhere to go.

## OBSERVABILITY

# Four families, and one trace with four readers

*An agent can be fast, return 200, and still be wrong, ungrounded, unaffordable or looping. Classic APM covers only the first row.*

### ● RED

*Is the service up?*

agent\_requests\_total  
agent\_request\_duration\_seconds  
agent\_errors\_total

### ● Agent

*Is it behaving?*

agent\_steps\_per\_turn  
agent\_tool\_calls\_total  
agent\_loop\_termination\_total

### ● Quality

*Is it right?*

agent\_groundedness\_ratio  
guardrail\_firings\_total  
agent\_eval\_score

### ● FinOps

*Is it affordable?*

llm\_cost\_usd\_per\_turn  
llm\_cache\_events\_total  
llm\_budget\_remaining\_usd

## One trace, four readers

### Engineering

a debugging artefact

### AI team

a quality signal

### Finance

a cost line

### The DPO

an audit record

PROMISES

# SLOs when the output is not deterministic

*You cannot promise any individual answer is right. You can promise a rate, measure it, and put an error budget behind it.*

| OBJECTIVE              | TARGET         | WINDOW | ERROR BUDGET |
|------------------------|----------------|--------|--------------|
| ● Gateway availability | 99.9%          | 30d    | 43.2 min     |
| ● Agent turn latency   | p95 < 4.0s     | 30d    | 43.2 min     |
| ● Groundedness         | ≥ 97% grounded | 7d     | 302.4 min    |
| ● Guardrail coverage   | 100% of turns  | 30d    | 0 min        |

PAGE

14.4× burn over 1 hour

2% of the 30-day budget gone within the hour. Somebody wakes up.

TICKET

6× burn over 6 hours

Degradation that will exhaust the budget this week. A single threshold catches one of these and misses the other.

# Four articles, four code paths

*A control that exists only in a document drifts. Each of these is a runtime object the service exposes.*

## Art. 50 Transparency

`guardrails.ensure_disclaimer`

Every reply carries an AI disclosure and a non-advice notice.

## Art. 12 Record-keeping

`telemetry.record_audit`

Append-only audit table, 7-year retention in the archive tier.

## Art. 14 Human oversight

`orchestrator HITL gate`

Side-effecting tools blocked behind an explicit human approval.

## Art. 15 Accuracy & robustness

`evals.suite.THRESHOLDS`

Groundedness scorer plus a CI eval gate at 0.95.

### Risk tier

## Limited risk

A customer-facing informational assistant. Not a medical device and not diagnostic, because every clinical judgement is routed to a registered pharmacist — which keeps it out of Annex III.

*The classification is a decision, made on the record. The moment the agent could give a dose, it is a different tier with a conformity assessment attached.*

GDPR: Art. 9 special-category data · basis 6(1)(b) + 9(2)(h) · EU-only residency enforced twice, by a Terraform variable validation and an OPA policy · redaction before the model call · transcripts 90 days, audit 7 years.

# Cost per turn is the number

*Everything else is a proxy for it. Four levers, in the order I would pull them.*

Illustrative — same traffic mix, levers applied in order



Measured per resolved conversation, not per call — an agent that is cheap per call and never resolves anything is expensive. The reflex to resist in a cost incident is “move everyone to a cheaper model”: that trades a cost problem you can see for a quality problem you cannot.

## 1 Tier routing

A greeting on the mini tier is ~16x cheaper per token than the flagship. The classifier that decides costs \$0.0001.

## 2 Semantic cache

“Where is my order”, “order status?” and “has it shipped” are one question. 25-35% hit rates are ordinary in support.

## 3 Bounded context

A rolling window stops prompt growth across a long session — the quiet cause of most cost regressions.

## 4 Hard ceilings

per request, per session, per tenant per day. A runaway loop stops itself, the gateway is the backstop.



# What is deliberately not here

*Each is a “not yet” with a stated trigger, not a “never”. Knowing what you are not building is most of platform product management.*

## Multi-agent orchestration framework

Two agents that need to coordinate usually mean one workflow was decomposed wrong. Earn it with a real case.

## Fine-tuning pipeline

Retrieval and prompting exhaust their headroom long before fine-tuning is the cheapest next move — and it brings a permanent lifecycle with it.

## Self-hosted models on GPUs

The economics only work at a volume Redcare does not have yet, and it moves a large operational burden onto a small team.

## A bespoke vector database

AI Search covers hybrid retrieval and is already inside the security perimeter.

## Kubernetes

Container Apps covers the requirements. AKS is a second product to operate, and the exit stays cheap because the unit of deployment is a plain OCI image.

## Agent-to-agent protocols

The standards are still moving. Committing early is expensive to undo.

# If you remember eight things

## ● The gateway is the whole thesis

It is the only path to a model, which is what makes budgets, entitlement, failover, caching, cost attribution and audit enforceable rather than advisory.

## ● Gate the side effect, not the reasoning

Read and draft freely; stop before changing a system of record. Art. 14 as a code path.

## ● Four signal families, not one

Up, behaving, right, affordable. A platform that ships only RED is trusted until the first bad answer.

## ● No public ingress, no unmonitored egress

And a private DNS zone for every private endpoint, or the traffic quietly leaves the VNet.

## ● Ordering is the argument

Budget before guardrails. Guardrails before the model. Approval before the side effect.

## ● Evals are the release gate

Tests prove the plumbing, evals prove the behaviour. Safety thresholded at 1.00.

## ● Region before provider

Failover keeps behaviour identical first. A provider switch changes behaviour and needs an eval run.

## ● Cost per resolved conversation

Not per call. An agent that is cheap per call and never resolves anything is expensive.